underlying infrastructure (typically how serverless computing works, but in this case for a container service) and enables faster time to market.

The EKS Distro, however, helps to create Kubernetes clusters anywhere with underlying open source Kubernetes components like kube-controller-manager, etcd, CoreDNS, kOps and Kubelet. This allows you to deploy the container image on-premise, in a private cloud, or any other cloud, including AWS EC2 or EKS.

A new public repository of container registry called Elastic Container Registry Public or ECR Public can be accessed from *https://gallery.ecr.aws/* with or without an AWS account to browse. It can be downloaded as a binary image for any container deployment. This turns ECR into a community initiative for storing, managing, sharing and also deploying container images, reducing the time spent reusing the images available in the public registry.

Cost factors in EKS

Effective cost management is crucial for any organisation utilising cloud-based container orchestration platforms like EKS. With EKS, achieving optimal cost efficiency involves understanding the various cost factors, implementing monitoring tools, and adopting best practices for resource utilisation. The major costs are as follows.

- ***Cluster costs:*** A fixed hourly rate is paid for each EKS cluster, regardless of usage.
- ***Node costs:*** EKS worker nodes run on standard EC2 instances, incurring regular EC2 costs based on instance type and usage.
- ***Networking and storage:*** These are costs associated with network traffic and the storage consumed by your EKS cluster.
- ***Operations tools:*** Additional costs may arise for services like Amazon Managed Service for Prometheus (AMP) used for monitoring.
- ***Third-party services:*** Any external tools or services integrated with EKS contribute to the overall cost.

Monitoring tools for EKS cost management

- ***Amazon Cost Management:*** This tool provides general cost visibility and usage reports for all AWS services, including EKS
- ***Kubecost:*** This open source tool is specifically designed for Kubernetes cost monitoring, offering detailed insights into resource allocation and cost attribution per namespace, cluster, and pod.
- ***Amazon Managed Service for Prometheus (AMP):*** Integrates with EKS to gather and analyse cluster metrics, enabling customised dashboards and cost analysis.

Best practices for EKS cost optimisation

- ***Right-sizing clusters:*** Choose the most appropriate EC2 instance type based on your workload needs, avoiding overprovisioning.
- ***Scaling on-demand instances:*** Leverage autoscaling groups to dynamically adjust resource allocation based on real-time demand.
- ***Utilising Spot Instances:*** Consider using Spot Instances for non-critical workloads to benefit from significant cost savings.
- ***Resource quotas and limits:*** Implement resource quotas and limits to prevent uncontrolled resource usage and cost spikes.
- ***Container optimisation:*** Optimise your container images and applications for minimal resource consumption.
- ***Automated shutdown:*** Implement automated shutdown processes for idle workloads to avoid unnecessary charges.
- ***Continuous monitoring and analysis:*** Regularly monitor your EKS resource usage and cost trends to identify potential savings opportunities.

Autoscalers in EKS

Autoscaling is a crucial aspect of managing EKS clusters efficiently. It ensures you have enough resources to run your workloads while avoiding unnecessary costs for idle resources. There are two main approaches to autoscaling in EKS.

Horizontal Pod Autoscaler (HPA)

Purpose: Scales the number of pods of a specific deployment based on resource metrics like CPU or memory usage.

Mechanism: Monitors the resource consumption of pods within a deployment and automatically scales the number of pods up or down to maintain the desired resource utilisation levels.

Benefits:

- Ensures application performance by scaling pods to meet demand.
- Reduces resource costs by scaling down pods when not needed.
- Easy to set up and configure.

Limitations:

- Can only scale pods within a single deployment.
- Doesn't factor in node resource availability.

Cluster Autoscaler

Purpose: Scales the number of worker nodes in your EKS cluster based on pod resource requests and availability.

Mechanism: Monitors the pods in the cluster and checks for pods that cannot be scheduled due to insufficient resources. If such pods exist, the Cluster Autoscaler automatically adds new nodes to the cluster. It also removes idle nodes to optimise resource utilisation.

Benefits:

- Scales the entire cluster based on overall resource needs.
- Supports heterogeneous node pools with different configurations.
- Integrates with various cloud providers like AWS and Azure.